ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) | 978-1-7281-6327-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICASSP49357.2023.10095125

PROMPT MAKES MASK LANGUAGE MODELS BETTER ADVERSARIAL ATTACKERS

He Zhu[*†], *Ce Li*[*†], *Haitian Yang*[*], *Yan Wang*[*], *Weiqing Huang*[*]

[*]Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[†]School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

ABSTRACT

Generating high-quality synonymous perturbations is a core challenge for textual adversarial tasks. However, candidates generated from the masked language model often contain many words that are antonyms or irrelevant to the original words, which limit the perturbation space and affect the attack's effectiveness. We present ProAttacker[1] which uses Prompt to make the mask language models better adversarial Attackers. ProAttacker inverts the prompt paradigm by leveraging the prompt with the class label to guide the language model to generate more semantically-consistent perturbations. We present a systematic evaluation to analyze the attack performance on 6 NLP datasets, covering text classification and inference. Our experiments demonstrate that ProAttacker outperforms state-of-the-art attack strategies in both success rate and perturb rate.

***Index Terms*—** Textual adversarial attack, mask language model, prompt

1. INTRODUCTION

Though the pre-train—fine-tune paradigm has become the de facto standard for many NLP tasks, the robustness of these neural models is often challenged by carefully crafted adversarial examples. That is, these examples can trigger incorrect predictions of the target model while they are imperceptible to humans. Adversarial examples provide a novel perspective for improving model robustness and interpreting models' decisions and have drawn much attention recently.

To produce natural, fluent, and grammatical outputs, recent studies [1–3] leverage the fill-masks mechanism of mask language model (MLM) to generate contextualized perturbations. However, contextualized perturbations are not always semantically-consistent perturbations; in other words, contextualized perturbations generated from the MLM can sometimes be antonyms or irrelevant to the original words. Understanding sample semantics and generating semantically consistent perturbation words is a huge challenge for MLM, especially when the original words are masked.

GPT-3 [4] reformulated the NLP tasks as fill-in-the-blanks problems with different prompts, which encouraged the pre-trained language model to generate task-specific words. Inspired by those findings, this work presents ProAttacker, a simple but efficient prompt-based adversarial attack method. ProAttacker leverages ground truth label-related prompt to enable MLM to generate perturbation words in a semantic class-aware manner. As illustrated in Figure 1, by adding label-related prompt and deploying mask tokens to the inputs, ProAttacker first reformulates input sentences into prompt cloze-style. Thus MLM generates candidates for MASK positions under the semantic guidance of prompt. Then ProAttacker ranks the vulnerable positions of the victim model and prioritizes filling them with the most semantically similar words until the prediction is altered.

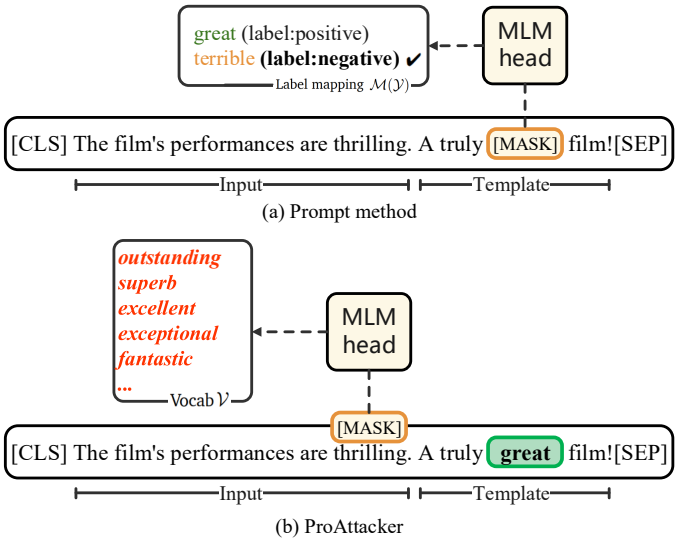

Fig. 1: An illustration of (a) Prompt method, (b) our proposed **ProAttacker**.

We evaluated ProAttacker on multiple datasets and corresponding fine-tuned BERT models. Experimental results show that ProAttacker successfully fooled those models with higher attack success rates and lower perturb rates compared with previous methods.

The main contributions of this study are as follows:

- We propose **ProAttacker**, a simple but efficient word-level adversarial method that employs label-related prompt to generate semantically consistent perturba-

[1]The source code of our attack can be found at `https://github.com/cgly/ProAttacker`

Authorized licensed use limited to: Concordia University Library. Downloaded on May 22,2025 at 00:05:43 UTC from IEEE Xplore. Restrictions apply.

tions.

- A novel prompt method is introduced as a general semantic enhancer for contextualized perturbation methods.
- Extensive experiments demonstrate the effectiveness of our model. We also conduct ablation studies and analyze the impact of different prompt templates and label words.

2. PROATTACKER

ProAttacker consists of three steps: (1) Prompt Construction (2) Vulnerable position detection (3) Performing Perturbations. We will describe each step in detail in this section.

2.1. Prompt Construction

The fill-in-the-blanks mode of GPT-3 aggregates the semantics of the whole input to predict its label. However, our ProAttacker inverts the paradigm by leveraging prompt with the class label to guide the fill-mask process.

Mask deployment ProAttacker first constructs a corrupted version set $\widetilde{\mathbf{X}}$ by deploy `[MASK]` on input $\mathbf{x}$ with two strategies: Replace(R), and Insert(I).

For a given position i of the input sequence, the Replace (R) operation is substituting the token x_i with `[MASK]`. Moreover, the Insert(I) operation inserts a `[MASK]` adjacent to x_i; it can be seen as adding additional information to the input. Denoted as:

$$\widetilde{\mathbf{X}}_R^i = x_1 \dots x_{i-1}\,\texttt{[MASK]}\,x_{i+1} \dots x_n \tag{1}$$

$$\widetilde{\mathbf{X}}_I^i = x_1 \dots x_i\,\texttt{[MASK]}\,x_{i+1} \dots x_n \tag{2}$$

To expand the potential perturbation space, ProAttacker performs the two operations described above for all possible positions. The corrupted version set $\widetilde{\mathbf{X}}$ can be expressed as:

$$\widetilde{\mathbf{X}} = \{\widetilde{\mathbf{X}}_R^i, \widetilde{\mathbf{X}}_I^i \mid 0 < i < |\mathbf{x}|\} \tag{3}$$

Prompt Addition ProAttacker fills the `[MASK]` slot in the prompt template with label word y that matches the "semantic class" and then fills slot <$\mathbf{x}$> with the corrupted version input text $\widetilde{\mathbf{x}}$, for each $\widetilde{\mathbf{x}} \in \widetilde{\mathbf{X}}$:

$$\mathcal{T}(\widetilde{\mathbf{x}}, y) = <\widetilde{\mathbf{x}}>, \text{A truly } [y] \text{ film!} \tag{4}$$

In this way, ProAttacker migrates the `[MASK]` slot from the template to the input and maintains the prompt form.

2.2. Vulnerable position detection

To further identify the vulnerable positions that significantly affect the final prediction results and then prioritize manipulating them, we utilize a scoring mechanism. The importance score $\mathbf{I}$ for a given mask position $\widetilde{\mathbf{x}}$ is measured by:

$$\mathbf{I}(\widetilde{\mathbf{x}}, y) = P_f(y \mid \mathbf{x}) - P_f(y \mid \widetilde{\mathbf{x}}_{z'}) \tag{5}$$

Algorithm 1 ProAttacker

Input: Text-label pair $(\mathbf{x}, y)$;
Input: Victim model f;
Input: prompt label word y;
Output: Adversarial example $\mathbf{x}_{adv}$
Initialization: $\mathbf{x}_{adv} = \mathbf{x}$
$\widetilde{\mathbf{X}} \leftarrow Prompt\ Construction(\mathbf{x})$
procedure VULNERABLE POSITION DETECTION
  for $\widetilde{\mathbf{x}}$ in $\widetilde{\mathbf{X}}$ **do**
    // The fill-mask process under prompt guidance
    $\mathcal{Z} \leftarrow \underset{z \in \mathcal{V}}{\text{Top}-k}\{P_{\mathcal{L}}([\text{ MASK }] = z \mid \mathcal{T}(\widetilde{\mathbf{x}}, y))\}$
    $\mathcal{Z} \leftarrow \{z \in \mathcal{Z} \mid \text{Sim}(\mathbf{x}, \widetilde{\mathbf{x}}_z) > \ell\}$
    // Determining the best perturbation word
    $z' \leftarrow \arg\max_{z \in \mathcal{Z}}\{P_f(y \mid \mathbf{x}) - P_f(y \mid \widetilde{\mathbf{x}}_z)\}$
    $\mathbf{I}(\widetilde{\mathbf{x}}, y) \leftarrow P_f(y \mid \mathbf{x}) - P_f(y \mid \widetilde{\mathbf{x}}_{z'})$
  $\widetilde{\mathbf{X}}' \leftarrow Sorted(\widetilde{\mathbf{X}})$ in descending order by $\mathbf{I}$
procedure PERFORMING PERTURBATIONS
  for $\widetilde{\mathbf{x}}$ in $\widetilde{\mathbf{X}}'$ **do**
    $\mathbf{x}_{adv} \leftarrow$ Apply z' on $\mathbf{x}_{adv}$
    if $f(\mathbf{x}_{adv}) \neq y$ **then return** $\mathbf{x}_{adv}$
  return NONE

where $P_f(y \mid \mathbf{x})$ is the probability of the ground truth label y predicted by victim model f given the input textual sequence $\mathbf{x}$. $\widetilde{\mathbf{x}}_z$ indicates that the `[MASK]` in $\widetilde{\mathbf{x}}$ is filled with perturbation word z. ProAttacker mine the perturbation words set $\mathcal{Z}$ from the mask language model $\mathcal{L}$. The candidate set of perturbation words can be represented as:

$$\mathcal{Z} = \underset{z \in \mathcal{V}}{\text{Top}-k}\{P_{\mathcal{L}}(\texttt{[MASK]} = z \mid \mathcal{T}(\widetilde{\mathbf{x}}, y))\} \tag{6}$$

where $P_{\mathcal{L}}$ denotes the output probability distribution of $\mathcal{L}$. The perturbation words $\mathcal{Z}$ should match the context that has been carefully modified by the prompt template $\mathcal{T}$ and label word y. To prevent the example's semantics from being distorted by antonyms, ProAttacker leverage the Universal Sentence Encoder (USE) [5] based sentence similarity scorer to filter the candidate set $\mathcal{Z}$ by:

$$\mathcal{Z} = \{z \in \mathcal{Z} \mid \text{Sim}(\mathbf{x}, \widetilde{\mathbf{x}}_z) > \ell\}. \tag{7}$$

Then, ProAttacker queries the victimization model to determine the best perturbation word z' with the biggest probability reducing for the ground truth class.

$$z' = \arg\max_{z \in \mathcal{Z}}\{P_f(y \mid \mathbf{x}) - P_f(y \mid \widetilde{\mathbf{x}}_z)\} \tag{8}$$

In contrast to previous work [1, 2, 6] that detected vulnerable positions by deleting words, our model directly compares the probability difference between the perturbed sentence and the original one. Thus, the scoring mechanism avoids semantic discrepancies caused by word deletion and is closer to the actual perturbation results.

Authorized licensed use limited to: Concordia University Library. Downloaded on May 22,2025 at 00:05:43 UTC from IEEE Xplore. Restrictions apply.

Task	Dataset	$\|\mathbf{Y}\|$	Len	Acc	Template	Label words
Classification	Yelp	2	152	98.7%	<$\mathbf{x}$>. All in all, it was [MASK].	great/terrible
	SST-2	2	10	93.2%	<$\mathbf{x}$>. A truly [MASK] film!	great/terrible
	AG News	4	43	95.6%	<$\mathbf{x}$>. It is a [MASK] News.	World/Sports/Business/Sci-Tech
	DBpedia	14	55	99.1%	<$\mathbf{x}$>. This topic is about [MASK].	Company/Education/Artist$\cdots$
Entailment	MNLI	3	23/11	94.0%	<$\mathbf{x}_1$>? [MASK],<$\mathbf{x}_2$>	Contradiction/Entailment/Neutral
	QNLI	2	11/31	97.6%	<$\mathbf{x}_1$>? [MASK],<$\mathbf{x}_2$>	Entailment/Not Entailment

Table 1: Overview of the datasets. $|\mathbf{Y}|$ signifies the number of classes. **Len** denotes the average sentence length. **Acc** is the accuracy of the example on the victimization model. Following are the manual prompt templates and label words.

2.3. Performing Perturbations

To produce adversarial examples with the least amount of modification, ProAttacker iteratively selects the perturbation position with a greedy algorithm.

For each remaining $\widetilde{\mathbf{x}} \in \widetilde{\mathbf{X}}$, we first rank the vulnerability scores $\mathbf{I}$ in descending order according to Eq.(5). The highest scoring position is selected from the remaining queue at each step. Perturbations are iteratively applied to the selected position until an adversarial example is found, or the perturbation queue is empty. Algorithm 1 summarizes the above procedure.

After each perturbation, the unmasked context changes, and the vulnerability scores may be inaccurate. At each step, we recalculate the best perturbation words z' based on the current context. However, vulnerable position detection requires traversing all potential positions. To simplify the computation, we assume the vulnerability scores are valid for the entire adversarial attack process.

3. EXPERIMENTS

3.1. Setup

Datasets and Victim models. We conduct a comprehensive evaluation across 6 NLP datasets, which covers text classification and natural language inference (Entailment).

- **Classification**: AG News [7],Yelp [7],SST-2 [8], DBPedia [7]
- **Entailment**: MNLI [9],QNLI [10]

To facilitate a fair comparison with the baselines, we evaluate our method in a set of 1000 examples that are open-sourced by the previous practices [3, 6]. Furthermore, considering the differences in victimization models, we employ the fine-tuned BERT models provided by the TextAttack [11] library. The dataset statistics and corresponding victim models' performances are summarized in Table 1.

Baselines. We compare ProAttacker with both enlightening and state-of-the-art black-box adversarial attack methods, including:

- **PWWS [12]**: It introduced a novel word salience ranking mechanism, then conducted a WordNet-based synonym swap.
- **TextFooler [6]**: A strong black box attack method employs Counter-fitted word embedding to find synonyms for each word.
- **CLARE [3]**: The state-of-the-art textual adversarial attack model. It greedily perturbs the original input with three kinds of operations.

The above baseline models were obtained from the open-source implementation provided by their authors.

Prompt Setting. We use the prompt templates and label words provided by the state-of-the-art prompts-based classification methods [13, 14]. Table I shows the prompt details. For MLM, we employ a lightweight MLM *Albert* [15] to conduct our major experiments.

Evaluation metrics.

Attack Success Rate (ASR): The rate of adversarial examples that can successfully attack the victim model.

Perturbed Rate (PR): The rate of the number of perturbed words to the input text length.

Average Query Number (Query): The average query number of launching a successful adversarial attack.

Perplexity (PPL): A common metric for evaluating the fluency of sentences. [2]

Semantic Similarity (Sim): The cosine similarity(USE [5]) between the input and its success attack examples.

3.2. Attacking Results

Table 2 summarizes the results. Although CLARE achieved significant improvement compared to the previous SOTA methods, ProAttacker still outperforms them in attack success rate, perturbed rate, and semantic similarity. We attribute this to the semantically consistent perturbations generated using the label-related prompt. In terms of perplexity, ProAttacker outperforms PWWS and Textfooler, but is slightly weaker than CLARE, partly because we adopt a more lightweight MLM. When it comes to the average query number metric, context-aware synonym replacement approaches (such as CLARE and ProAttacker) perform less effectively. This is primarily because context-aware perturbation words must be obtained by querying the mask language model with a greedy strategy for every perturbation.

[2] https://huggingface.co/docs/transformers/perplexity

Authorized licensed use limited to: Concordia University Library. Downloaded on May 22,2025 at 00:05:43 UTC from IEEE Xplore. Restrictions apply.

Dataset	**Yelp** (Acc=0.983, PPL = 56.3)					**SST-2** (Acc=0.925, PPL = 113.955)					**QNLI** (Acc=0.976, PPL = 47.3)				
Model	**ASR↑**	**PR↓**	**Query↓**	**PPL↓**	**Sim↑**	**ASR↑**	**PR↓**	**Query↓**	**PPL↓**	**Sim↑**	**ASR↑**	**PR↓**	**Query↓**	**PPL↓**	**Sim↑**
PWWS	46.08	8.11	**152**	101	0.612	42.43	11.50	**49**	192	0.559	13.42	**8.60**	**69**	83	0.634
TextFooler	32.96	15.94	816	145	0.789	55.95	17.58	245	319	0.585	52.86	16.35	314	168	0.672
CLARE	43.84	7.38	639	**77**	0.798	74.81	10.17	477	**170**	0.670	64.03	12.02	811	**80**	0.727
ProAttacker	**47.40**	**7.03**	646	88	**0.803**	**80.39**	**9.82**	463	188	**0.674**	**65.16**	11.62	820	94	**0.730**
Dataset	**AG News** (Acc=0.925, PPL = 72.0)					**DBpedia** (Acc=0.99, PPL = 51.1)					**MNLI** (Acc=0.94, PPL = 60.8)				
Model	**ASR↑**	**PR↓**	**Query↓**	**PPL↓**	**Sim↑**	**ASR↑**	**PR↓**	**Query↓**	**PPL↓**	**Sim↑**	**ASR↑**	**PR↓**	**Query↓**	**PPL↓**	**Sim↑**
PWWS	15.56	8.51	**95**	143	0.656	7.89	7.88	**118**	89	0.544	21.46	**6.09**	**88**	103	0.667
TextFooler	50.91	22.66	537	328	0.635	33.95	22.62	711	169	0.646	52.49	12.70	360	153	0.722
CLARE	52.21	6.07	2178	**97**	0.690	46.40	**7.74**	971	**70**	0.694	67.97	7.73	708	**84**	0.787
ProAttacker	**57.83**	**5.72**	1985	120	**0.718**	**50.60**	8.54	1041	77	**0.697**	**72.66**	7.25	679	89	**0.789**

Table 2: Evaluation results of attacking against various fine-tuned BERT models on 1000 test instances. We report the five key metrics mentioned above and signify the best performance of all attacks in black font.

Module	**ASR↑**	**PR↓**	**Query↓**	**PPL↓**	**Sim↑**	**Ant↓**
ProAttacker	**80.39**	9.82	**463**	188	**0.674**	**40.25**
w/o Template $\mathcal{T}$	77.54	10.06	472	189	0.673	48.81
w/o Label words	77.66	10.09	480	194	0.671	49.50
w/o Prompt	76.05	10.07	473	187	0.673	48.18
CLARE	74.81	10.17	477	170	0.670	55.62
CLARE+Prompt	76.79	**9.79**	465	**168**	0.665	43.37

Table 3: Ablation study results.

Template	**Label words**	**Metrics**	
SST-2 (positive/negative)		ASR	PR
<**x**> A truly `[MASK]` film!	Positive/Negative	82.60	10.48
<**x**> A truly `[MASK]` film!	great/terrible	81.52	10.45
<**x**> A truly `[MASK]` film!	good /bad	**83.69**	**10.04**
<**x**> A truly `[MASK]` film!	Yes/No	79.34	10.55
<**x**> A truly `[MASK]` film!	inspiring/weird	79.34	10.07
<**x**> A truly `[MASK]` film!	good/bad	**83.69**	10.04
<**x**> So `[MASK]`.	good/bad	81.52	9.73
<**x**> And that is `[MASK]`.	good/bad	79.34	**9.01**
<**x**> The ending is `[MASK]`.	good/bad	78.26	9.56
<**x**> I thought it was `[MASK]`.	good/bad	79.34	9.63

Table 4: The impact of templates and label words.

4. ANALYSIS AND DISCUSSION

4.1. Ablation Study

To verify the effectiveness of each component, we performed ablation studies on the above-mentioned SST-2 dataset with 1000 examples. To measure perturbation words quality, we examine the size of perturbation words candidate set $\mathcal{Z}$ after semantic consistency filtering (Eq.7). The USE similarity threshold is set to 0.8 and **Ant(%)** is used to indicate the rate of antonyms or irrelevant words that are filtered in this process. The results are summarized in Table 3.

The Impact of Prompt Obviously, blank label words(*w/o* Label words) and blank templates(*w/o* Template) lead to a worse attack performance. This indicates templates and label words should be considered as a whole, and ProAttacker still needs templates as the "semantic bridge". We further examined the effectiveness of the prompt on different attack methods. With the prompt component, the attack success rate of ProAttacker and CLARE increased by 4.34% and 1.98%, whereas the perturbation rate decreased by 0.25% and 0.38%, respectively.

The quality of perturbation words The evaluation shows the percentage of antonyms and irrelevant words in the candidate words set decreased from 48.18% to 40.25% under the guidance of prompt. In other words, the prompt enhances language models to generate more semantically-related perturbations, which expands the perturbation space and thus improves the attack efficiency.

4.2. Prompt Engineering

We freeze other parameters and settings and validate the performance of different templates and label word pairs on 100 instances of SST-2.

As shown in Table 4 Both label words and templates significantly affect the performance of adversarial attacks. The label words(good/bad) achieve the best attack success rate and modification rate at the same time. Meanwhile, label words provided by the dataset are not optimal, e.g. Negative/Positive does not achieve the best performance.

5. CONCLUSION

In this paper, we propose ProAttacker, a simple but efficient prompt-based adversarial attack method. ProAttacker leverages prompt to enable MLM to generate perturbation words in a semantic class-aware manner, generating semantically consistent perturbations. Our work also demonstrates that prompt can be a general semantically consistent enhancer for contextualized perturbations methods. Extensive experiments show the effectiveness of ProAttacker and its advantages over baselines on different NLP tasks.

Authorized licensed use limited to: Concordia University Library. Downloaded on May 22,2025 at 00:05:43 UTC from IEEE Xplore. Restrictions apply.

6. REFERENCES

[1] Linyang Li, Ruotian Ma, Qipeng Guo, Xiangyang Xue, and Xipeng Qiu, "Bert-attack: Adversarial attack against bert using bert," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6193–6202.

[2] Siddhant Garg and Goutham Ramakrishnan, "Bae: Bert-based adversarial examples for text classification," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6174–6181.

[3] Dianqi Li, Yizhe Zhang, Hao Peng, Liqun Chen, Chris Brockett, Ming-Ting Sun, and William B Dolan, "Contextualized perturbation for textual adversarial attack," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021, pp. 5053–5069.

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al., "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[5] Daniel Cer, Yinfei Yang, Sheng-yi Kong, Nan Hua, Nicole Limtiaco, Rhomni St John, Noah Constant, Mario Guajardo-Cespedes, Steve Yuan, Chris Tar, et al., "Universal sentence encoder," *arXiv preprint arXiv:1803.11175*, 2018.

[6] Di Jin, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits, "Is bert really robust? natural language attack on text classification and entailment," *arXiv preprint arXiv:1907.11932*, 2019.

[7] Xiang Zhang, Junbo Zhao, and Yann LeCun, "Character-level convolutional networks for text classification," in *Proc. of NeurIPS*, 2015.

[8] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. of EMNLP*, 2013.

[9] Adina Williams, Nikita Nangia, and Samuel Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proc. of NAACL*, 2018.

[10] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman, "Glue: A multi-task benchmark and analysis platform for natural language understanding," in *Proc. of ICLR*, 2019.

[11] John Morris, Eli Lifland, Jin Yong Yoo, Jake Grigsby, Di Jin, and Yanjun Qi, "Textattack: A framework for adversarial attacks, data augmentation, and adversarial training in nlp," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 119–126.

[12] Shuhuai Ren, Yihe Deng, Kun He, and Wanxiang Che, "Generating natural language adversarial examples through probability weighted word saliency," in *Proceedings of the 57th annual meeting of the association for computational linguistics*, 2019, pp. 1085–1097.

[13] Tianyu Gao, Adam Fisch, and Danqi Chen, "Making pre-trained language models better few-shot learners," *arXiv preprint arXiv:2012.15723*, 2020.

[14] Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang, "Gpt understands, too," *arXiv preprint arXiv:2103.10385*, 2021.

[15] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," *CoRR*, vol. abs/1909.11942, 2019.

Authorized licensed use limited to: Concordia University Library. Downloaded on May 22,2025 at 00:05:43 UTC from IEEE Xplore. Restrictions apply.